

EMBEDDED C PROGRAMING

```
#include <stdint.h>
#include <stdio.h>

int main() {
    // 1. Declare a 32-bit unsigned integer (register)
    uint32_t register_val = 0x00000000;

    // 2. Define the bit position (5th bit = index 5)
    int bit_position = 5;

    // 3. Set the bit using OR operator and a bitmask
    // Mask: (1<< 5) results in 00000000 00000000 00000000 00100000 (0x20)
    register_val |= (1U << bit_position);

    printf("Updated Register: 0x%08X\n", register_val);
    return 0;
}
```

```
Register value: 0x00000020

=== Code Execution Successful ===
```

```
</stdio.h></stdint.h>
```

```
#include <stdio.h>
#include <stdint.h>

// Using a macro for efficiency
#define TOGGLE_BIT(reg, pos) (reg ^= (1U << pos))

int main() {
    uint8_t register_val = 0x00; // Binary: 00000000

    // Toggle bit index 6 (the 7th bit)
    TOGGLE_BIT(register_val, 6);

    printf("Register after toggle: 0x%02X\n", register_val); // Output: 0x40
    return 0;
}

</stdint.h></stdio.h>
```

```
Register after toggle: 0x40

=== Code Execution Successful ===
```

```

#include <stdio.h>

int main() {
    int num;
    // The position to check (0-indexed: 4th bit is 2^4)
    int pos = 4;

    printf("Enter an integer: ");
    scanf("%d", &num);

    // Create a mask by shifting 1 left by 'pos' positions
    // If (num & mask) is non-zero, the bit is set
    if (num & (1<< pos)) {
        printf("The 4th bit (position %d) is SET (1).\n", pos);
    } else {
        printf("The 4th bit (position %d) is NOT SET (0).\n", pos);
    }

    return 0;
}

```

Output

```

Enter an integer: 12
The 4th bit (position 4) is NOT SET (0).

```

```

=== Code Execution Successful ===

```

```

} </stdio.h>

```

```
#include <stdio.h>
```

```
/**
```

```
 * Counts set bits by clearing the rightmost set bit in each iteration.
```

```
 * Time Complexity:  $O(k)$ , where  $k$  is the number of set bits.
```

```
 */
```

```
int countSetBits(unsigned int n) {
```

```
    int count = 0;
```

```
    while (n > 0) {
```

```
        n &= (n - 1); // Clears the rightmost set bit
```

```
        count++;
```

```
    }
```

```
    return count;
```

```
}
```

```
int main() {
```

```
    unsigned int num = 15; // Binary: 1111
```

```
    printf("Number of set bits in %u is %d\n", num, countSetBits(num));
```

```
    return 0;
```

```
}
```

```
Number of set bits in 15 is 4
```

```
=== Code Execution Successful ===
```

```
</stdio.h>
```

```
#include <stdio.h>
```

```
int main() {
```

```
    unsigned int num;
```

```
    int shift;
```

```
    // User input for the number and shift count
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%u", &num);
```

```
    printf("Enter number of positions to shift: ");
```

```
    scanf("%d", &shift);
```

```
    // Left Shift Operation: num << shift
```

```
    // Effectively multiplies 'num' by 2 raised to the power of 'shift'
```

```
    unsigned int leftShifted = num << shift;
```

```
    // Right Shift Operation: num >> shift
```

```
    // Effectively divides 'num' by 2 raised to the power of 'shift'
```

```
    unsigned int rightShifted = num >> shift;
```

```
    // Displaying the results
```

```
    printf("\nOriginal number: %u\n", num);
```

```
    printf("Result after Left Shift (<< %d): %u\n", shift, leftShifted);
```

```
    printf("Result after Right Shift (>> %d): %u\n", shift, rightShifted);
```

```
    return 0; </stdio.h>
```

```
Enter a positive integer: 33
Enter number of positions to shift: 5
```

```
Original number: 33
Result after Left Shift (<< 5): 1056
Result after Right Shift (>> 5): 1
```

```
=== Code Execution Successful ===
```

```
#include <stdio.h>

int main() {
    int a, b;

    // Prompt user for input
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    printf("\nBefore swapping: a = %d, b = %d\n", a, b);

    // XOR Swap Algorithm
    a = a ^ b; // a now holds the XOR of both original values
    b = a ^ b; // b now holds original value of a
    a = a ^ b; // a now holds original value of b

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

```
Enter two integers: 1,2

Before swapping: a = 1, b = -4240688
After swapping:  a = -4240688, b = 1

=== Code Execution Successful ===
```

```
</stdio.h>
```

```
#include <stdio.h>
```

```
int main() {  
    unsigned int num = 13; // Binary: 1101  
    int pos = 2;  
  
    // Create mask and apply AND  
    unsigned int result = num & (1<< pos);  
  
    printf("Original: %u\n", num);  
    printf("Result after clearing: %u\n", result); // Output: 4  
    return 0;  
}
```

```
Original: 13  
Result after clearing: 4  
  
=== Code Execution Successful ===
```

```
</stdio.h>
```